

COMM114: Transformer as an AR model

Guoqiang Zhang

November 2025

Recap from last lecture

Modeling Sequences via Probability

- Suppose the joint distribution of the terms in sequences is characterized by $P_{data}(\{Y_i\}_{i=1}^n)$.
 - Assumption: Each Y_i is a discrete random variable, taking values from a discrete set Ω .
 - For example, for natural language processing, the discrete set includes all the possible words or subwords.
- It is known from probability theory that the joint distribution can be decomposed as

$$P_{data}(\{Y_i\}_{i=1}^n) = P_{data}(Y_1) \prod_{i=2}^n P_{data}(Y_i | \{Y_{k < i}\})$$

where $\{Y_{k < i}\}$ includes all the random variables up to index i .

Remark: For simplicity, we only consider discrete random variables. One can also extend the analysis to continuous case.

Recap from last lecture (cont.)

Auto-Regressive (AR) Models

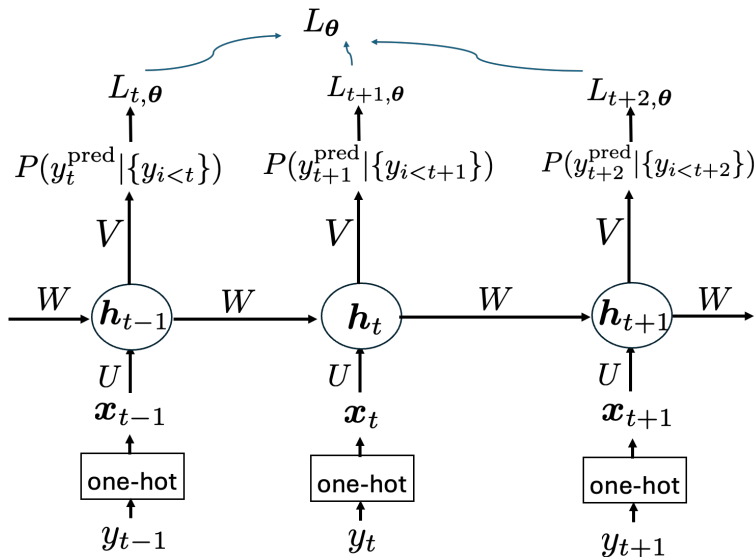
- An AR model $P_{\theta}(\{Y_i\}_{i=1}^n)$ attempts to learn the conditional distribution $P_{data}(Y_i|\{Y_{k<i}\})$ for all index i .
- Once an AR model is trained, it can be used to generate new sequences in a recurrent manner, one term per timestep according to the learned conditional probability.

One-hot vector pre-processing:

- Let $\mathbf{y} = [y_1, y_2, \dots, y_n]$ be a particular sequence drawing from the data distribution $P_{data}(\{Y_i\}_{i=1}^n)$.
- Converting each y_i into a one-hot vector $\mathbf{x}_i \in \mathbb{R}^d$.

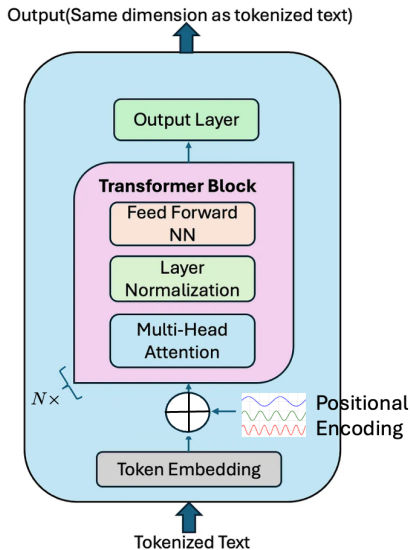
Recap from last lecture

Exploiting RNN for learning the conditional distribution:



Transformer

- In the **training stage**, Transformer processes all the terms in a sequence in parallel while RNN needs to process the terms in a sequence sequentially.
- In the **inference stage**, Transformer generates one term per step, just like RNN.



Tokenization

- Pre-processing a text sequence into a sequence of token indices.
- The token indices are alternatively referred to *tokenized IDs*.
- The tokenized IDs are fed to the Transformer for further processing.

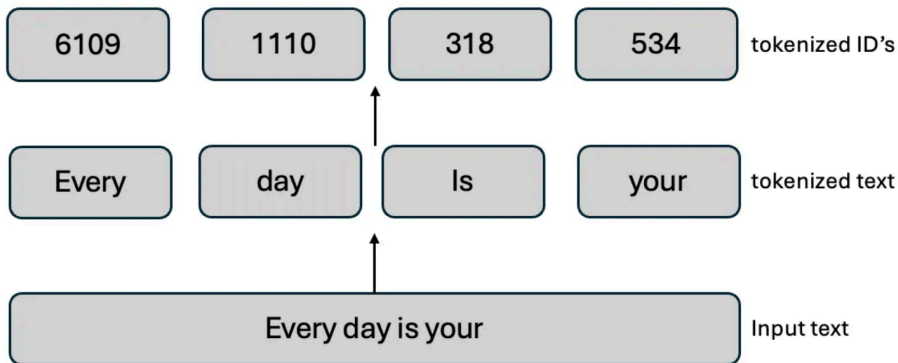
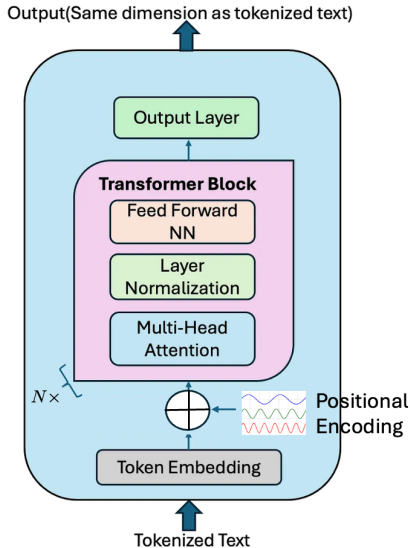


Figure: Tokenization by BPE method

Token Embedding

- Token-Embedding is responsible for transforming the input tokenized-IDs into continuous numerical vectors (referred to as *embeddings*) that the model can process.
- In other words, Token-Embedding attempts to map high-dimensional categorical variables (e.g., tokenized-IDs) to a low-dimensional learned representation (i.e., continuous vectors) that places similar entities closer together in the embedding space
- In practice, a Token-Embedding layer is realized by a lookup table that stores all the continuous vectors for all the elements in a vocabulary.



Token Embedding (cont.)

- An example of Pytorch code

```
# Embedding Layer
class Embedding(nn.Module):
    def __init__(self, vocab_size, embed_size):
        super().__init__()
        # Initialize the embedding layer with the specified
        # vocabulary size and embedding dimension

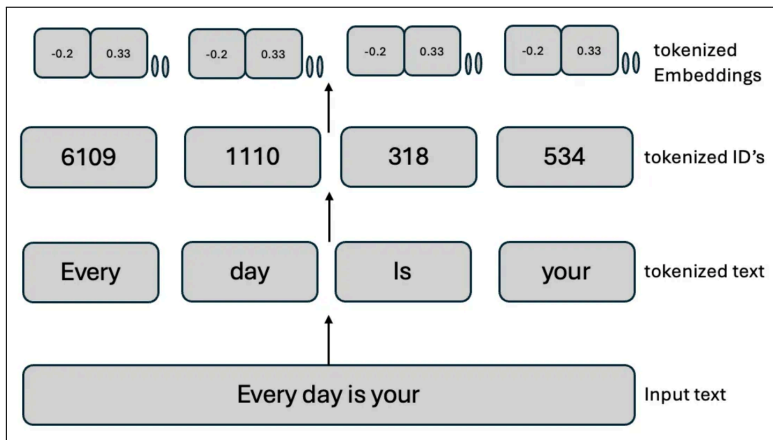
        self.embed = nn.Embedding(vocab_size, embed_size)

    def forward(self, x):
        # Forward pass: convert input token IDs
        # to their corresponding embeddings

        return self.embed(x)
```

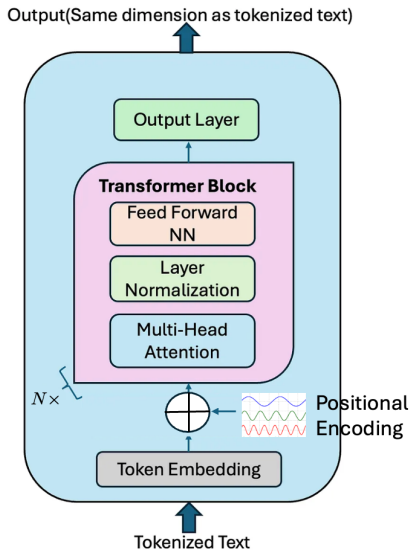
[Remark website-link](#) to learn more about torch.nn.Embedding

Token Embedding (cont.)

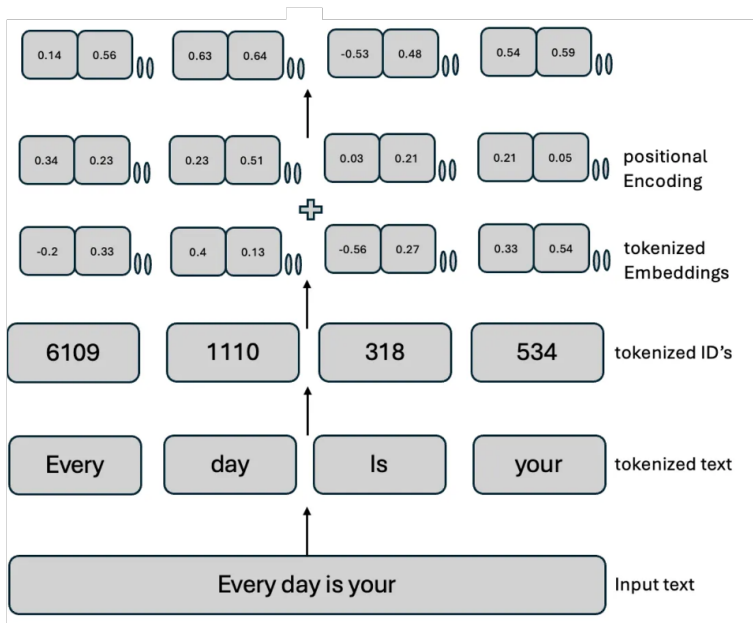


Positional Encoding

- Pos.-Encoding adds sinusoidal signals to the tokenized embeddings.
- The frequencies of the sinusoidal signals are different across different token positions in a sequence.
- Objective of Pos.-Encoding
 - To provide the model with information about the order of tokens in a sequence.
 - Different from RNNs, Transformer does not have a built-in way to understand the positions of the tokens.
 - In the training process, Transformer architecture process all tokens in parallel.



Positional Encoding (cont.)



Positional Encoding (cont.)

- An example of Pytorch code

```
# Positional Encoding
class PositionalEncoding(nn.Module):
    def __init__(self, embed_size, max_seq_length=512):
        super().__init__()
        # Initialize a tensor to hold the positional encodings
        pe = torch.zeros(max_seq_length, embed_size)

        # Create a tensor for positions (0 to max_seq_length)
        position = torch.arange(0, max_seq_length, dtype=torch.float).unsqueeze(1)

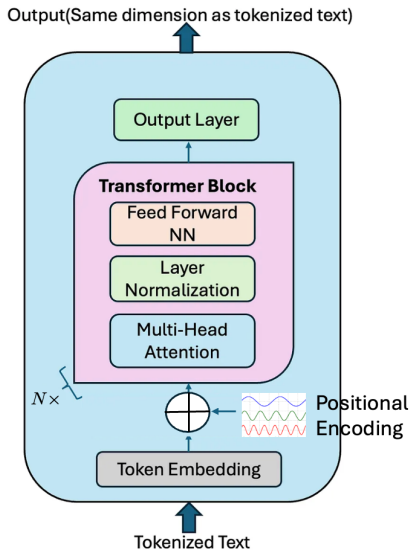
        # Calculate the division term for the sine and cosine functions
        div_term = torch.exp(torch.arange(0, embed_size, 2).float() * -(math.log(10000.0) / embed_size))

        # Apply sine to even indices and cosine to odd indices
        pe[:, 0::2] = torch.sin(position * div_term) # Sine for even indices
        pe[:, 1::2] = torch.cos(position * div_term) # Cosine for odd indices

        # Register the positional encodings as a buffer (not a model parameter)
        self.register_buffer('pe', pe.unsqueeze(0)) # Shape: (1, max_seq_length, embed_size)
    def forward(self, x):
        # Add the positional encodings to the input embeddings
        return x + self.pe[:, :x.size(1)]
```

Multi-Head Attention

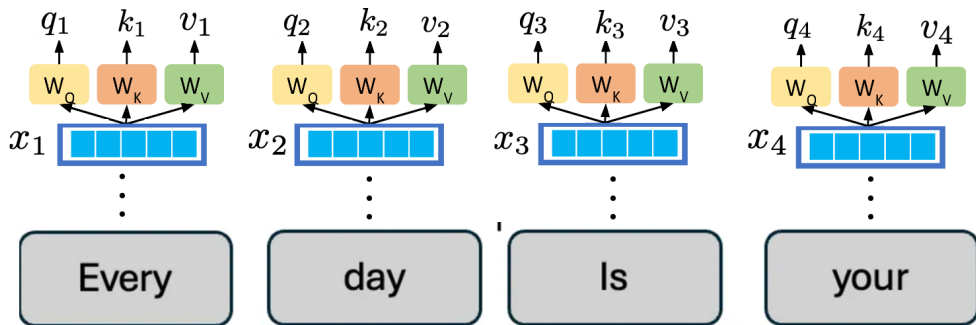
- The attention mechanism first gathers information from past tokens and then updates the current token accordingly.
- The attention mechanism computes the relative correlation of the current token with all the past tokens. The past tokens which are highly correlated will contribute more when updating the current token.
- The reason we only consider **past tokens** is that we are treating **Transformer as an auto-regressive (AR) model**.
- If we use future tokens for prediction, then information-leakage occurs.



Single-Head Attention

- We first focus on single-head attention to demonstrate the basic idea.
- Multi-head attention is a straightforward extension of single-head attention.
- **Step 1:** Compute the three vectors (q_i, k_i, v_i) for each token x_i .

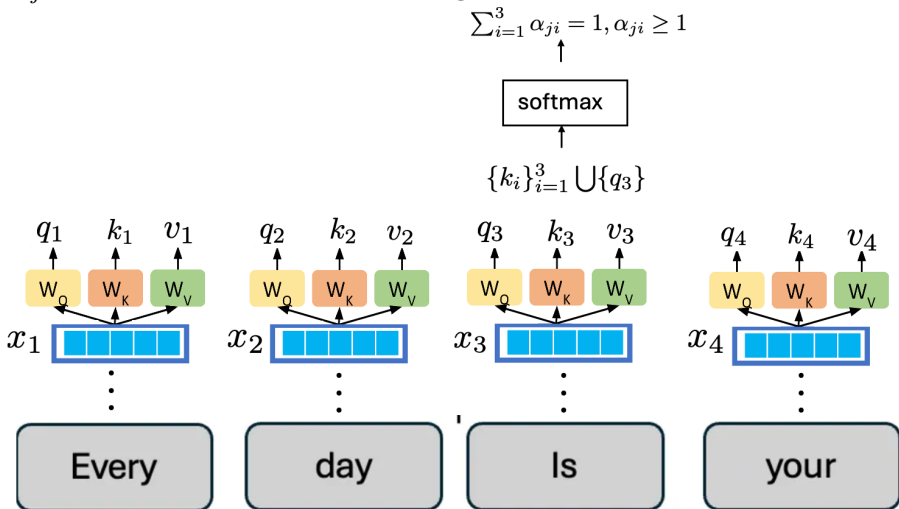
$$q_i = x_i W_q \quad k_i = x_i W_k \quad v_i = x_i W_v \quad [(W_q, W_k, W_v) \text{ are shared across all tokens}]$$



- $\{q_i\}$: queries; $\{k_i\}$: keys; $\{v_i\}$: values
- We assume they are all row-vectors.

Single-Head Attention (cont.)

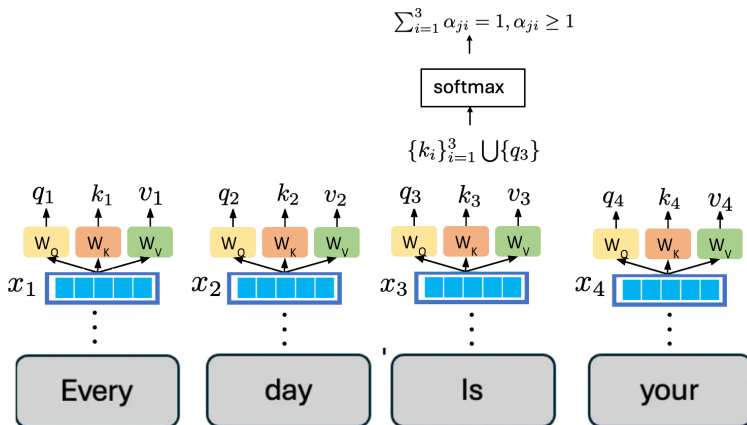
- **Step 2:** Compute the relative correlations of all past tokens with respect to the current token i .
- $1 \geq \alpha_{ji} \geq 0$ indicates the correlation strength between token j and token i .



Single-Head Attention (cont.)

- **Step 2:** Compute the relative correlations of all past tokens with respect to the current token i . [d_h : dimension of q_i or k_i .]

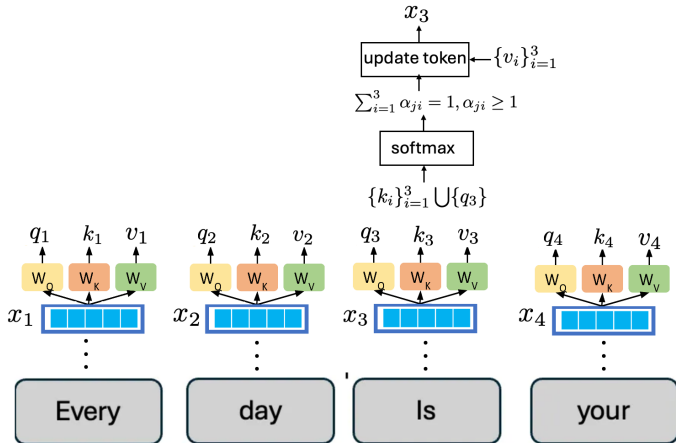
$$\{\alpha_{ji}\}_{j=1}^3 = \text{softmax}(\{\langle k_j, q_3 \rangle\}_{j=1}^3 / \sqrt{d_h}) \quad [\langle \cdot \rangle \text{denote inner product of two vectors}]$$



Single-Head Attention (cont.)

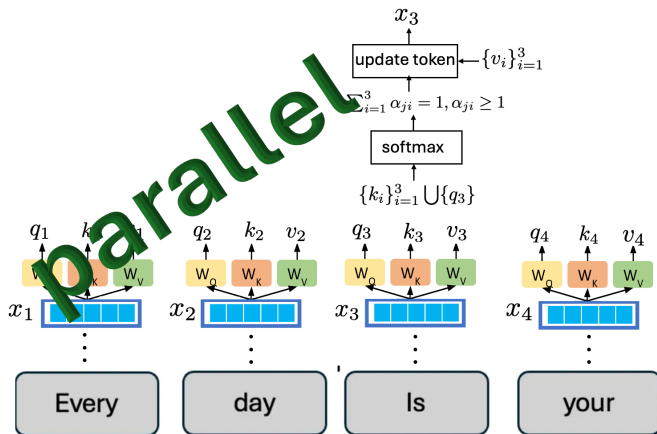
- Step 3: Update token x_i

$$x_3 = x_3 + \left(\sum_{j=1}^3 \alpha_{ji} v_j \right) W_o$$



Single-Head Attention (cont.)

- All the tokens $\{x_i\}_{i=1}^4$ can be updated in parallel in the training process.



Single-Head Attention (cont.)

- We use X to denote the tensor that includes all the token (row-wise) vectors $\{x_i\}_{i=1}^n$.

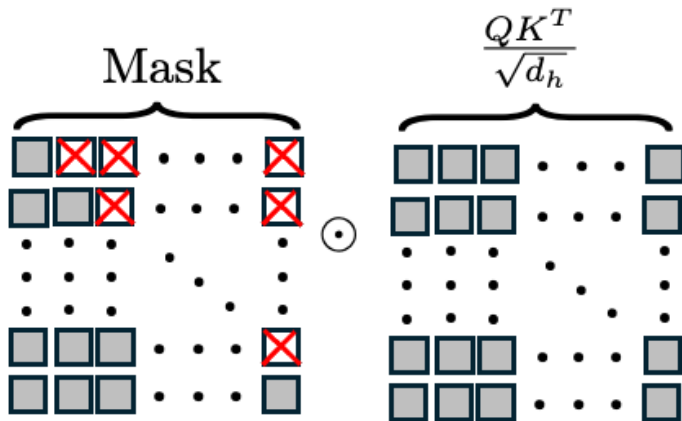
$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n \times d} \quad [n : \text{number of tokens}, d : \text{dimension per token vector}]$$

- The update for all the tokens in a single-head attention can be written as a compact form:

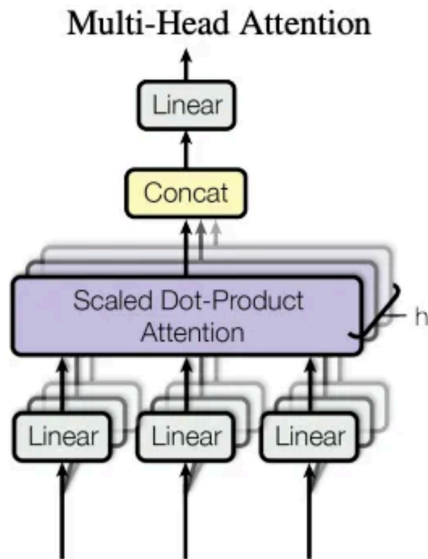
$$\begin{aligned} H(X) &= \text{mask-attention}(\overbrace{XW_Q}^Q, \overbrace{XW_K}^K, \overbrace{XW_V}^V) \\ &= \text{softmax} \left(\text{Mask} \odot \frac{QK^T}{\sqrt{d_h}} \right) V \\ X &\Leftarrow X + H(X)W_o \end{aligned}$$

Single-Head Attention (cont.)

- The mask is imposed to the matrix $\frac{QK^T}{\sqrt{d_h}}$ to make sure future tokens would not contribute the update of the current token.
- $\odot$ denotes element-wise operation.
- The elements with $\boxed{\times}$ are assigned with an $-\infty$ value.



Multi-Head Attention



Multi-Head Attention (cont.)

- The update expression for a multi-head attention can be rerepresented into a compact form as

$$H_m(X) = \text{mask-attention}(\overbrace{XW_m^Q}^{Q_m}, \overbrace{XW_m^K}^{K_m}, \overbrace{XW_m^V}^{V_m}) \quad m = 1, \dots, M$$

info. aggregation

$$= \text{softmax} \left(\text{Mask} \odot \frac{Q_m K_m^T}{\sqrt{d_m}} \right) V_m$$

$$\text{MH}(X) = \text{Concat}(H_1(X), \dots, H_M(X))$$

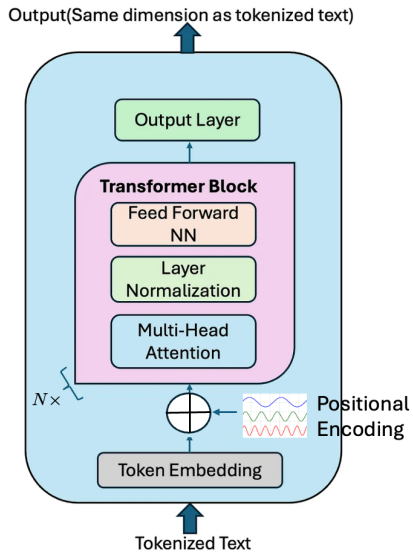
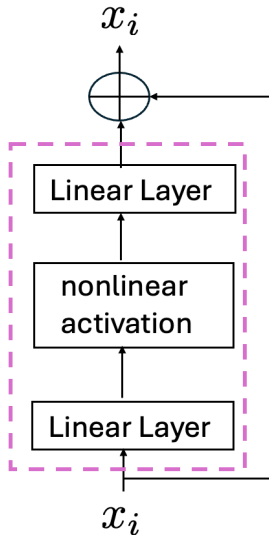
linear mapping

$$X \Leftarrow X + \overbrace{\text{MH}(X)W^o}$$

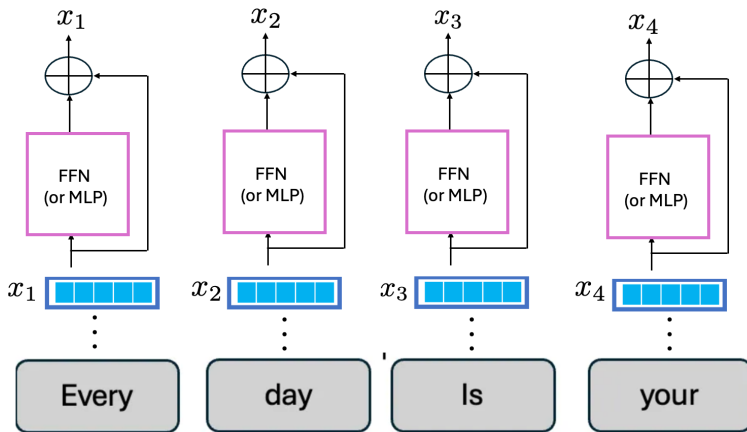
skip-connection

Feed-Forward NN

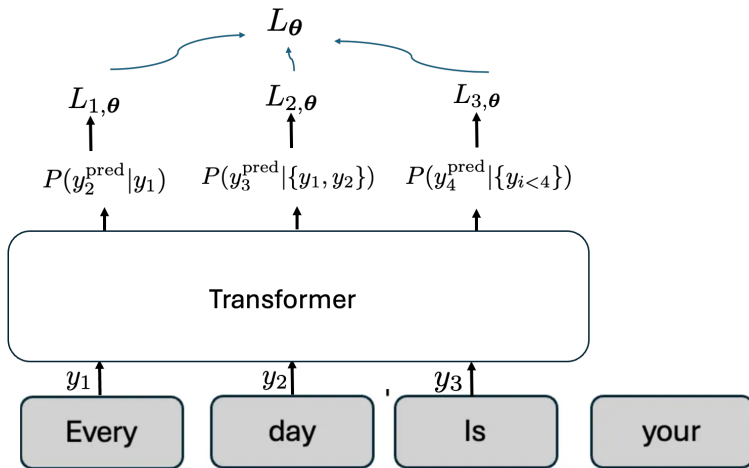
- The FFN (which is in fact an MLP) is used to further process the attention outputs and capture more complex transformations.



Feed-Forward NN (cont.)



Training Transformer



References

- "From Theory to Code: Step-by-Step Implementation and Code Breakdown of GPT-2 model"
by Vipul Koti
- Lecture notes by Liangze Li and Kateryna Shapovalenko

Questions?